

✖ What is `numeric_limits` in C++?

`numeric_limits` is a **template class** in the C++ Standard Library (found in the `<limits>` header file). It helps you find the **properties and limits of data types** — like the largest or smallest value they can hold.

To use it, you must include this line at the top of your program:

```
#include <limits>
```

💡 What does `numeric_limits<int>::max()` mean?

Let's read it piece by piece:

```
numeric_limits<int>::max()
```

Part	Meaning
<code>numeric_limits</code>	A class template that provides information about data types.
<code><int></code>	Tells the compiler that we want information specifically for <code>int</code> (integer data type).
<code>::</code>	The scope resolution operator , used to access members of a class or namespace.
<code>max()</code>	A function (or constant) that returns the maximum value that this data type (<code>int</code>) can hold.

🔍 Example Usage

```
#include <iostream>
#include <limits>
using namespace std;

int main() {
    cout << "Maximum value of int: " << numeric_limits<int>::max() << endl;
    return 0;
}
```

Output:

```
Maximum value of int: 2147483647
```

💡 How It Works Internally

- The `int` type typically uses **4 bytes (32 bits)** of memory.

- One bit is used for the **sign** (positive/negative).
- That leaves **31 bits** for the value itself.
- The maximum positive value is: [$2^{\{31\}} - 1 = 2,147,483,647$]
- The minimum value is: [$-2^{\{31\}} = -2,147,483,648$]

So:

```
numeric_limits<int>::max() // 2147483647
numeric_limits<int>::min() // -2147483648
```

⚙ Other Useful Functions in `numeric_limits`

You can use similar syntax for other properties:

Function	Description	Example Output
<code>numeric_limits<int>::min()</code>	Smallest value (negative for int)	<code>-2147483648</code>
<code>numeric_limits<int>::lowest()</code>	Same as <code>min()</code> for integers	<code>-2147483648</code>
<code>numeric_limits<int>::digits</code>	Number of binary digits used	<code>31</code>
<code>numeric_limits<float>::max()</code>	Largest possible float	<code>3.40282e+38</code>
<code>numeric_limits<double>::min()</code>	Smallest positive double	<code>2.22507e-308</code>

✅ In Short

`numeric_limits<int>::max()` → gives you the **largest possible integer value** that can be stored in your computer's memory for the `int` type.

Perfect 👍 Here's a clear and beginner-friendly **table comparing `numeric_limits::max()` and `numeric_limits::min()`** values for the most common C++ data types.

📊 Table: Numeric Limits of Basic C++ Data Types

Data Type	Bytes (Typical)	Minimum Value (<code>min()</code>)	Maximum Value (<code>max()</code>)	Description
<code>bool</code>	1	<code>false (0)</code>	<code>true (1)</code>	Stores true/false values
<code>char</code>	1	<code>-128</code>	<code>127</code>	Used to store single characters (like <code>'A'</code> , <code>'@'</code>)

Data Type	Bytes (Typical)	Minimum Value (min())	Maximum Value (max())	Description
unsigned char	1	0	255	Stores only positive characters
short	2	-32,768	32,767	Small integer type
unsigned short	2	0	65,535	Only positive small integers
int	4	-2,147,483,648	2,147,483,647	Standard integer type
unsigned int	4	0	4,294,967,295	Only positive integers
long	4 or 8	-2,147,483,648 or smaller	2,147,483,647 or larger	Longer integer type (depends on compiler)
unsigned long	4 or 8	0	4,294,967,295 or larger	Positive long integers
long long	8	-9,223,372,036,854,775,808	9,223,372,036,854,775,807	Very large integers
float	4	1.17549e-38	3.40282e+38	Single-precision decimal number
double	8	2.22507e-308	1.79769e+308	Double-precision decimal number
long double	12 or 16	3.3621e-4932	1.18973e+4932	Higher precision floating-point number

💡 Important Notes for Beginners

1. The exact values may vary slightly depending on your **compiler** and **system architecture (32-bit or 64-bit)**.
2. **unsigned** types **cannot store negative numbers**, but they can store **larger positive numbers**.
3. For **floating-point numbers (float, double)**, `min()` gives the **smallest positive value**, not the most negative one.
 - To get the **most negative** value, use `numeric_limits<float>::lowest()`.
4. Always include this header when using numeric limits:

```
#include <limits>
```

📌 Example: Displaying All Data Type Limits in Code

Here's a short program to display some of these values:

```
#include <iostream>
#include <limits>
using namespace std;

int main() {
    cout << "int: " << numeric_limits<int>::min() << " to " <<
    numeric_limits<int>::max() << endl;
    cout << "float: " << numeric_limits<float>::min() << " to " <<
    numeric_limits<float>::max() << endl;
    cout << "double: " << numeric_limits<double>::min() << " to " <<
    numeric_limits<double>::max() << endl;
    cout << "char: " << (int)numeric_limits<char>::min() << " to " <<
    (int)numeric_limits<char>::max() << endl;
    return 0;
}
```

Output (example):

```
int: -2147483648 to 2147483647
float: 1.17549e-38 to 3.40282e+38
double: 2.22507e-308 to 1.79769e+308
char: -128 to 127
```